

Agent Evaluation Lab

Deployment Manual

May 2026

1. Purpose

This manual explains how to install, run, validate, and troubleshoot Agent Evaluation Lab.

It is written for:

- team members who need to run the project locally
- reviewers who need a reliable reproduction path
- instructors or evaluators who want to verify the end-to-end workflow

The instructions are based on the current repository structure and the code paths that are present in this project today.

2. What This Project Includes

Agent Evaluation Lab has three main runtime parts:

1. Python evaluation framework

Includes the CLI, scenario runner, grading, policy checks, reports, and runner service.

2. Built-in RAG target agent

A local HTTP agent used for demo and end-to-end testing.

3. Next.js platform

Includes the sandbox, dashboard, evaluation workflow, release gate, evidence downloads, regression features, and monitoring views.

3. Local Demo Architecture

For a standard local demo, the project runs on these ports:

- 8000 - built-in RAG agent
- 3000 - Next.js platform

- 5433 - PostgreSQL database via Docker

4. Prerequisites

Install the following before starting:

- Python 3.11 or newer
- Node.js 18 or newer
- npm
- Docker Desktop
- Git
- An OpenAI API key

Optional but useful:

- uv for Python environment management
- PostgreSQL client tools

5. Clone the Repository

```
git clone https://github.com/tejakusireddy/agent-eval-lab.git
cd agent-eval-lab
```

Optional verification:

```
git status --short --branch
```

6. Configure Environment Files

6.1 Root Python Environment

Create the root environment file:

```
cp .env.example .env
```

Edit `.env` and set at least:

```
OPENAI_API_KEY=sk-your-real-key
RAG_MODEL=gpt-4o-mini
RAG_TEMPERATURE=0.1
RAG_MAX_TOKENS=512
RAG_HOST=127.0.0.1
RAG_PORT=8000
```

You can also export the API key directly in the terminal:

```
export OPENAI_API_KEY="sk-your-real-key"
```

6.2 Platform Environment

Create the platform environment file:

```
cd platform
cp .env.example .env
cd ..
```

For local development with the included Docker PostgreSQL service, set:

```
DATABASE_URL="postgresql://agent_eval:agent_eval_password@localhost:5433/agent_eval?schema=public"
OPENAI_API_KEY=sk-your-real-key
PYTHON_CLI_PATH=../.venv/bin/python
AGENT_EVAL_PATH=../agent_eval_lab
RBAC_DEFAULT_ROLE=admin
RBAC_STRICT=false
```

```
PUBLIC_SELF_SERVE_MODE=false
```

Notes:

- Placeholder Clerk values may remain unchanged for local development.
- The application falls back to development authentication outside production.
- Do not commit `.env` or `platform/.env`.

7. Install Dependencies

7.1 Recommended Python Setup

This project requires Python 3.11+.

Check your Python version:

```
python3.11 --version
```

Create and activate a virtual environment:

```
python3.11 -m venv .venv  
source .venv/bin/activate
```

Install the project in editable mode:

```
python -m pip install -e "[dev]"
```

7.2 Alternative Python Setup with `uv`

If `uv` is installed:

```
uv sync
source .venv/bin/activate
```

7.3 Platform Dependencies

Install Node dependencies:

```
cd platform
npm install
cd ..
```

8. Start the Local Database

The repository includes a Docker Compose file for local PostgreSQL.

Start only the database service:

```
cd platform
docker compose up -d postgres
```

Verify the container:

```
docker compose ps
```

Generate Prisma client and apply the schema:

```
npm run db:generate
npm run db:push
```

```
cd ..
```

Expected result:

- Prisma Client is generated
- the `agent_eval` schema is applied successfully

Important note:

`platform/docker-compose.yml` also includes Redis and worker-related services, but the repository does not currently include the root Dockerfile needed for all worker containers. For the standard local demo, start **PostgreSQL only**.

9. Start the Application Locally

Open three terminal windows or tabs.

Terminal 1: Start the Built-In RAG Agent

From the repository root:

```
source .venv/bin/activate
export OPENAI_API_KEY="sk-your-real-key"
python -m agent_eval_lab.rag_service.server
```

Verify health:

```
curl -i http://127.0.0.1:8000/health
```

Expected result:

- HTTP 200 OK

Open in browser:

- `http://127.0.0.1:8000/`

Terminal 2: Start the Platform

```
cd platform  
npm run dev
```

Open in browser:

- `http://127.0.0.1:3000`

Terminal 3: Optional CLI Verification

From the repository root:

```
source .venv/bin/activate  
export OPENAI_API_KEY="sk-your-real-key"  
agent-eval run-all-scenarios --config-file  
examples/rag_agent_config.yaml
```

Generated reports are written to:

- `reports/`

10. Run the Platform Demo

10.1 Test the Target Agent Directly

Open:

- `http://127.0.0.1:8000/`

Submit a simple prompt such as:

- `What is Agent Evaluation Lab?`

This confirms the target agent is working before platform-based evaluation begins.

10.2 Test Agent Connectivity in the Sandbox

Open:

- `http://127.0.0.1:3000/sandbox`

Use:

- Base URL: `http://127.0.0.1:8000`

Then:

1. Run auto-detect
2. Run health check
3. Send a test query

Expected result:

- health check succeeds
- the agent returns an answer
- query output is visible in the platform

10.3 Run a New Evaluation

Open:

- `http://127.0.0.1:3000/dashboard/evaluations/new`

Recommended flow:

1. Select HTTP agent
2. Use base URL `http://127.0.0.1:8000`
3. Select a scenario set
4. Start the evaluation
5. Open the evaluation detail page

Expected result:

- status moves through `queued`, `running`, and then `completed` or `failed`
- scenario-level outcomes become visible
- reports can be downloaded after completion

10.4 Download Artifacts

From the evaluation detail page, verify the following downloads:

- JSON report
- Markdown report
- evidence artifact

11. Validation Commands

Run these before submission, demo, or deployment handoff.

11.1 Python Tests

```
source .venv/bin/activate
pytest
```

11.2 Platform Build

```
cd platform
npm run build
cd ..
```

11.3 CLI Smoke Run

```
source .venv/bin/activate
export OPENAI_API_KEY="sk-your-real-key"
agent-eval run-all-scenarios --config-file
examples/rag_agent_config.yaml
```

11.4 Release Gate Validation

```
source .venv/bin/activate
agent-eval check-gate \
  --report-file reports/evaluation_report.json \
  --policy-file policy/release_gate.enterprise.yaml
```

12. Production Deployment Model

The recommended production architecture separates the web platform from the long-running evaluator.

12.1 Deploy the Platform

Deploy the `platform/` directory to Vercel.

Required environment variables:

```
DATABASE_URL=postgresql://...
OPENAI_API_KEY=sk-...
EVAL_RUNNER_URL=https://your-runner-service.example.com/v1/evaluate
EVAL_RUNNER_TOKEN=replace-with-shared-secret
EVAL_EXECUTION_MODE=inline
API_KEY_HASH_SALT=replace-with-random-hex
```

Optional public self-serve settings:

```
PUBLIC_SELF_SERVE_MODE=true  
PUBLIC_SELF_SERVE_DAILY_LIMIT=3  
PUBLIC_SELF_SERVE_MAX_SCENARIOS=3  
PUBLIC_SELF_SERVE_SALT=replace-with-random-hex
```

Optional audit and evidence settings:

```
SLACK_WEBHOOK_URL=  
JIRA_WEBHOOK_URL=  
AUDIT_SIGNING_KEY=replace-with-long-random-secret  
AUDIT_SIGNING_KEY_ID=audit-2026-q1  
EVIDENCE_SIGNING_KEY=replace-with-long-random-secret  
EVIDENCE_SIGNING_KEY_ID=key-2026-q1
```

After deployment, apply the Prisma schema against the production database from a trusted terminal:

```
cd platform  
DATABASE_URL="postgresql://..." npm run db:push  
cd ..
```

12.2 Deploy the Evaluator Runner

Deploy the Python runner service on a long-running host such as Render, Fly.io, Railway, or a VM.

Start command:

```
uvicorn agent_eval_lab.runner.service:app --host 0.0.0.0 --port  
8080
```

Required environment variables:

```
OPENAI_API_KEY=sk-...  
EVAL_RUNNER_TOKEN=replace-with-shared-secret  
SCENARIO_DEFINITIONS_DIR=scenario_definitions
```

Health check:

```
curl https://your-runner-service.example.com/health
```

12.3 Connect the Platform to the Runner

Set these values in the platform deployment:

```
EVAL_RUNNER_URL=https://your-runner-service.example.com/v1/evaluate  
EVAL_RUNNER_TOKEN=replace-with-shared-secret
```

The shared token must match the runner service configuration.

13. Troubleshooting

13.1 Python Version Error

Symptom:

- install fails because the system default Python is below 3.11

Fix:

```
python3.11 --version  
python3.11 -m venv .venv  
source .venv/bin/activate
```

```
python -m pip install -e ".[dev]"
```

13.2 `pip: command not found`

Fix:

```
python3.11 -m pip install -e ".[dev]"
```

13.3 Port `8000` Already In Use

Check the existing process:

```
lsof -i :8000
```

Stop it:

```
kill <PID>
```

Then start the RAG service again.

13.4 Platform Cannot Reach the Database

Symptom:

- Prisma reports `Can't reach database server at localhost:5433`

Fix:

```
cd platform
```

```
docker compose up -d postgres
npm run db:push
docker compose ps
cd ..
```

13.5 Sandbox Health Check Fails

Verify:

- the RAG service is running
- `OPENAI_API_KEY` is present
- base URL is `http://127.0.0.1:8000`

Test directly:

```
curl -i http://127.0.0.1:8000/health
```

13.6 Platform Styling or Static Assets Look Broken

Clean and restart the dev server:

```
cd platform
npm run dev:clean
cd ..
```

Then hard refresh the browser.

13.7 Evaluation Remains Queued or Running

Check:

- the target agent is reachable
- PostgreSQL is running
- the API key is configured

- the evaluation page is refreshing normally

Retry from the UI if the failure was transient.

13.8 Report Download Fails

Reports are available only after a run has completed or stored output exists.

To test the endpoint directly:

```
curl -i
"http://127.0.0.1:3000/api/evaluations/{id}/download?format=json"
```

Replace `{id}` with a real evaluation ID.

14. Security Notes

- Never commit `.env` or `platform/.env`
- Rotate any key that appears in screenshots or shared logs
- Use `EVAL_RUNNER_TOKEN` to protect the remote runner
- Use `API_KEY_HASH_SALT` in production
- Use evidence and audit signing keys for review-grade artifact integrity

15. Quick Command Reference

Root Setup

```
git clone https://github.com/tejakusireddy/agent-eval-lab.git
cd agent-eval-lab
cp .env.example .env
python3.11 -m venv .venv
source .venv/bin/activate
python -m pip install -e "[dev]"
```

Platform Setup

```
cd platform
cp .env.example .env
npm install
docker compose up -d postgres
npm run db:generate
npm run db:push
npm run dev
```

Start the Target Agent

```
cd ..
source .venv/bin/activate
export OPENAI_API_KEY="sk-your-real-key"
python -m agent_eval_lab.rag_service.server
```

Validate the System

```
source .venv/bin/activate
pytest
agent-eval run-all-scenarios --config-file
examples/rag_agent_config.yaml
cd platform && npm run build
```

16. Maintainer Notes

Before submission or handoff:

- run the validation commands
- capture the screenshots listed in Section 15
- confirm `.env`, `platform/.env`, generated reports, `.next`, `node_modules`, and local exports are not staged
- confirm README and deployment instructions match the current repository behavior